

Application Layer Protocols

Comprehensive Notes on E-Mail Architecture (SMTP, POP3, IMAP) & DNS

Computer Networks Study Series

July 23, 2026

Contents

1	Electronic Mail Architecture	2
1.1	Core Components of the E-Mail System	2
1.2	Simple Mail Transfer Protocol (SMTP)	2
1.2.1	Key Characteristics of SMTP	2
1.2.2	End-to-End Email Delivery Flow	3
1.3	Comparison: HTTP vs. SMTP	3
1.4	Email Access Protocols: POP3 vs. IMAP vs. HTTP	4
1.4.1	Webmail (HTTP / HTTPS)	4
1.4.2	POP3 vs. IMAP Technical Comparison	4
1.4.3	Pros, Cons, and Real-World Scenarios	4
2	Domain Name System (DNS)	6
2.1	Why Not a Centralized DNS?	6
2.2	The Distributed Hierarchical DNS Structure	6
2.3	DNS Name Resolution Methods	7
2.3.1	1. Iterative Query (Standard Internet Practice)	7
2.3.2	2. Recursive Query	7
2.4	DNS Caching and TTL	7
2.5	DNS Resource Records (RR)	8

1 Electronic Mail Architecture

The Electronic Mail system is a distributed application that allows users to exchange messages asynchronously across the Internet.

1.1 Core Components of the E-Mail System

An e-mail network consists of three primary high-level components:

1. **User Agent (UA):** The software application that the end-user interacts with directly to compose, read, reply to, and forward email messages.
 - *Desktop Clients:* Microsoft Outlook, Mozilla Thunderbird, Apple Mail.
 - *Webmail Clients:* Gmail, Yahoo Mail, Outlook Web (accessed via browser).
2. **Mail Server:** The core backbone infrastructure responsible for storing, forwarding, and routing email messages. Every user is associated with a mail server.
 - **Mailbox:** A dedicated storage area on the mail server holding incoming messages for a specific user.
 - **Outgoing Message Queue:** A buffer queue where outgoing emails reside before being transmitted to the destination mail server.
3. **Simple Mail Transfer Protocol (SMTP):** The standard application layer protocol used to transfer mail between servers and from sending user agents to servers.

1.2 Simple Mail Transfer Protocol (SMTP)

SMTP Definition & Function

SMTP operates on **TCP Port 25**. It is a **push protocol** used exclusively to push emails from a sender's client to their local mail server, and between mail servers across the Internet.

Crucial Note: SMTP is **NOT** used by a receiving user agent to pull or fetch emails from its own server's mailbox.

1.2.1 Key Characteristics of SMTP

- **Persistent Connections:** SMTP can keep a TCP connection open to transfer multiple email messages sequentially between the same two mail servers, reducing connection overhead.
- **Two-Tier Handshaking Process:** First, a standard **TCP 3-Way Handshake** (SYN → SYN-ACK → ACK) establishes the transport channel. Then, an **SMTP Application-Level Handshake** occurs where the client and server exchange greetings (HELO/EHLO) and authenticate.
- **Legacy ASCII Requirement:** Originally, SMTP required all message data (headers and body) to be encoded as 7-bit ASCII text. Binary attachments must be encoded using standards like MIME (Multipurpose Internet Mail Extensions).

1.2.2 End-to-End Email Delivery Flow

Step-by-Step Scenario: Alice sends email to Bob

1. **Composition:** Alice uses her User Agent (e.g., Thunderbird) to draft an email addressed to bob@yahoo.com.
2. **Client-to-Server Transfer:** Alice's User Agent uses **SMTP** over TCP Port 25 (or 587) to send the message to her local Mail Server (smtp.gmail.com).
3. **Queuing & Routing:** Alice's mail server places the message into its **Outgoing Queue**.
4. **DNS Lookup:** Alice's server looks up the Mail Exchange (MX) DNS record for yahoo.com to get the IP address of Bob's mail server.
5. **Server-to-Server Transfer:** Alice's mail server initiates a TCP connection to Bob's mail server (mail.yahoo.com) on Port 25 and transfers the message using **SMTP**.
6. **Mailbox Storage:** Bob's mail server receives the email and places it into Bob's specific inbox/mailbox.
7. **Retrieval:** Bob opens his User Agent. The agent connects to Bob's server using an **Email Access Protocol** (POP3, IMAP, or HTTP/HTTPS) to retrieve the message.

1.3 Comparison: HTTP vs. SMTP

Both HTTP and SMTP are application layer protocols used for transferring data over TCP/IP, but they serve fundamentally different interaction models.

Table 1: Detailed Architectural Comparison between HTTP and SMTP

Feature	HTTP (Hypertext Transfer Protocol)	SMTP (Simple Mail Transfer Protocol)
Data Transfer Mode	PULL Protocol: The client requests (pulls) data from the server.	PUSH Protocol: The sending server sends (pushes) data to the receiving server.
Primary Role	Client-to-Server web document retrieval.	Server-to-Server (and Client-to-Server) message pushing.
Default Port	Port 80 (HTTP) / Port 443 (HTTPS)	Port 25 (Server-to-Server SMTP)
Data Formatting	Unconstrained binary data transferred directly.	Originally strict 7-bit ASCII (requires MIME for binary attachments).
Encapsulation	Each object (image, text) is requested and sent in its own separate HTTP response.	Multipart messages: entire email including multiple attachments is bundled in a single request.
Commands/Codes	ASCII commands; numeric status codes (e.g., 200 OK, 404 Not Found).	ASCII commands; numeric status codes (e.g., 250 OK, 550 Mailbox Unavailable).

1.4 Email Access Protocols: POP3 vs. IMAP vs. HTTP

Since SMTP is strictly a push protocol, a recipient cannot use SMTP to pull mail from their remote mailbox. Dedicated **Email Access Protocols** perform this retrieval function.

1.4.1 Webmail (HTTP / HTTPS)

When accessing mail via web browsers (e.g., gmail.com), HTTPS acts as the retrieval protocol between the user agent (browser) and the mail server. Server-to-server delivery still relies entirely on SMTP.

1.4.2 POP3 vs. IMAP Technical Comparison

Table 2: Comparative Matrix: POP3 vs. IMAP

Feature	POP3 (Post Office Protocol v3)	IMAP (Internet Message Access Protocol)
Default Port	Port 110 (Unencrypted) / Port 995 (SSL)	Port 143 (Unencrypted) / Port 993 (SSL)
Mail Location	Downloads mail to local client and (typically) deletes it from server.	Keeps all messages centrally stored on the server.
Multi-Device Access	Poor. Synced status across multiple devices is not supported.	Excellent. Synchronizes states (read, deleted, flagged) across all devices seamlessly.
Server State Update	Does not allow organizing or managing server folders/mailboxes.	Allows full manipulation: create folders, update flags, delete messages on server.
Readability	Must download the entire message before reading.	Supports partial downloading (e.g., fetch headers or text only without heavy attachments).
Security / Virus Risk	Mails reside on local workstation; local machine is vulnerable to virus vectors.	Mails stay on server; server security controls apply (less local disk risk).

1.4.3 Pros, Cons, and Real-World Scenarios

POP3 (Post Office Protocol):

- **Pros:** Low server storage requirements; emails are accessible offline once downloaded; fast reading speed for local storage.
- **Cons:** Email state is tied to one device; loss of local hard drive means complete loss of emails; difficult to manage across modern multi-device environments.
- **Ideal Scenario:** Desktop workstation with limited internet connection bandwidth and strict server storage quotas, accessed exclusively from one physical office computer.

IMAP (Internet Message Access Protocol):

- **Pros:** Centralized storage; state changes reflect everywhere; lightweight preliminary downloading (e.g., previewing message subject before downloading a 50MB PDF attachment).
- **Cons:** Heavy reliance on persistent connection; higher server storage utilization costs.
- **Ideal Scenario:** Modern workflow where a user accesses their inbox across a smartphone, tablet, laptop, and web client simultaneously.

2 Domain Name System (DNS)

Computers route packets over networks using IP addresses (e.g., 142.250.190.46), whereas human beings prefer memorable alphanumeric names (e.g., google.com). The **Domain Name System (DNS)** serves as the Internet's directory service to map hostnames to IP addresses.

2.1 Why Not a Centralized DNS?

A centralized single-server model for DNS is impossible for several core reasons:

1. **Single Point of Failure:** If the central server crashes, the entire global Internet effectively halts.
2. **Enormous Traffic Volume:** A single server cannot process billions of queries per second.
3. **Distant Geographic Location:** Physical distance causes massive latency for remote users worldwide.
4. **Maintenance Scalability:** Managing millions of daily updates in a single database is un-maintainable.

2.2 The Distributed Hierarchical DNS Structure

To achieve massive scalability, DNS uses a distributed, hierarchical database structure.

Hierarchy Order

Root DNS Servers → Top-Level Domain (TLD) Servers → Authoritative DNS Servers

1. Root DNS Servers:

- Positioned at the top of the hierarchy.
- There are **13 logical Root Server addresses** globally (named `a.root-servers.net` through `m.root-servers.net`), managed by 12 independent organizations (e.g., ICANN, NASA, Verisign).
- Every logical address is backed by hundreds of physical servers using *Anycast* routing.

2. Top-Level Domain (TLD) Servers:

- Responsible for generic top-level domains (`.com`, `.org`, `.edu`, `.net`) and country-code domains (`.us`, `.uk`, `.bd`).

3. Authoritative / Second-Level Domain (SLD) Servers:

- Maintained by organizations or domain hosts (e.g., Google, Amazon).
- Houses the actual mappings of human-readable hostnames to IP addresses for public resources.
- **Rule:** The combination of SLD + TLD (e.g., google + `.com`) **MUST be globally unique**.

4. Local Name Server (DNS Resolver):

- Does not strictly belong to the formal hierarchy, but plays a crucial proxy role.
- Managed by an ISP, university, or corporate network (e.g., Google Public DNS 8.8.8.8).
- Acts as the immediate point-of-contact for end-user queries.

2.3 DNS Name Resolution Methods

When a client requests a domain name resolution, two main query traversal strategies exist:

2.3.1 1. Iterative Query (Standard Internet Practice)

In an **iterative query**, the queried server replies with the best answer it has or a *referral* pointing to the next name server in the hierarchy. The **Local DNS Resolver** bears the burden of executing all subsequent contacts.

Iterative Query Resolution Path

1. Host contacts **Local DNS Resolver** for `mail.example.com`.
2. Local DNS checks cache; if miss, queries **Root DNS Server**.
3. Root replies: *"I don't know the IP, but here is the IP for the .com TLD Server."*
4. Local DNS queries **.com TLD Server**.
5. TLD Server replies: *"I don't know the IP, but here is the Authoritative Server for example.com."*
6. Local DNS queries **Authoritative DNS Server** (`example.com`).
7. Authoritative Server returns the exact IP address for `mail.example.com`.
8. Local DNS caches the result and returns it to the host client.

2.3.2 2. Recursive Query

In a **recursive query**, the client requests the server to resolve the address completely on its behalf. The recipient server forwards the request to the next server and waits for the final response before returning it back down the chain.

- **Drawback:** Places heavy computational and query tracking loads on upper-tier root and TLD servers. Consequently, Internet root and TLD servers typically disable recursive queries for public users.

2.4 DNS Caching and TTL

To reduce network traffic and response latency, DNS relies heavily on caching.

- **Caching Mechanism:** Once a Local Name Server resolves any hostname mapping, it caches the mapping locally in memory. If another host asks for a host in the same domain, the Local Name Server can respond immediately.
- **Time-To-Live (TTL):** Every cached record contains a **TTL** value (in seconds) set by the authoritative server. After TTL expires, the cache entry is discarded to prevent stale data.
- **Out-of-Date Cache Issues:** If a website changes its IP address while local caches still hold unexpired TTL records, traffic will briefly route to the old IP until the TTL expires or cache is flushed.

2.5 DNS Resource Records (RR)

DNS servers store mapping entries in a standardized format called ****Resource Records (RRs)****.

Resource Record Format

RR Format = (**Name**, **Value**, **Type**, **TTL**)

The meaning of **Name** and **Value** depends on the record **Type**:

Table 3: Primary DNS Resource Record Types

Type	Name	Value	Description / Example
Type A	Hostname	IPv4 Address	Maps a hostname to its IPv4 address.
	<i>google.com</i>	<i>172.217.16.196</i>	
Type NS	Domain	Authoritative Hostname	Specifies authoritative server hostname for domain.
	<i>google.com</i>	<i>ns1.google.com</i>	
Type MX	Domain	Mail Server Hostname	Specifies mail server responsible for domain.
	<i>google.com</i>	<i>mail.google.com</i>	
Type CNAME	Alias Name	Canonical (Real) Name	Maps an alias domain name to its real name.
	<i>www.google.com</i>	<i>google.com</i>	

MX and A Record Chaining Dependency

To send an email to `user@google.com`:

1. The sending server performs an **MX Record lookup** for `google.com`.
2. The DNS server returns: (`google.com`, `mail.google.com`, `MX`).
3. Since `mail.google.com` is a hostname (not an IP address), the sending server must then perform a **Type A lookup** for `mail.google.com` to obtain the actual IP address (`172.10.12.39`) needed to establish TCP Port 25 connection.